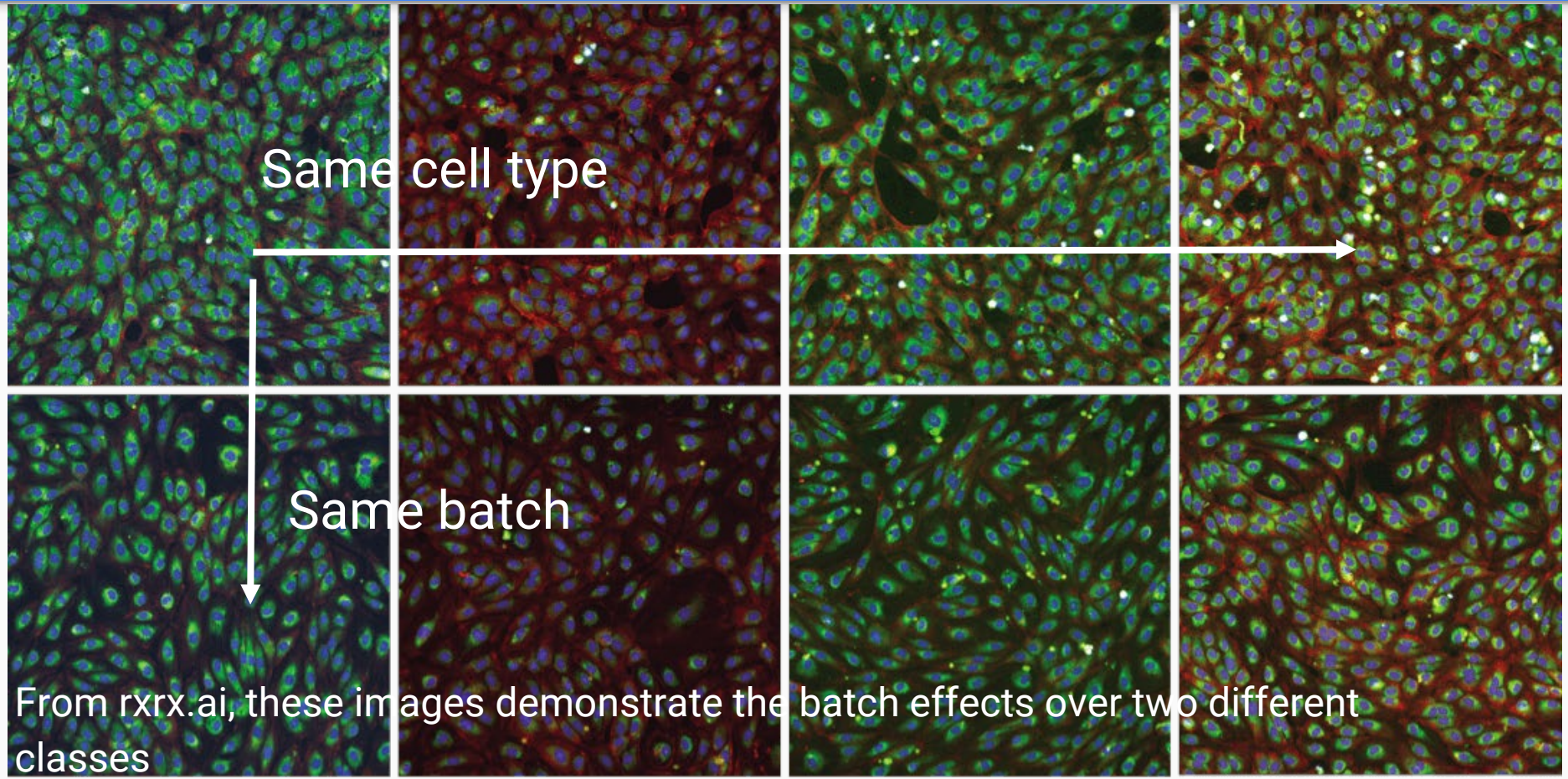
Self Supervised Learning on Human Cancer Cellular Images

Starting Motivation: Kaggle *Recursion* Challenge

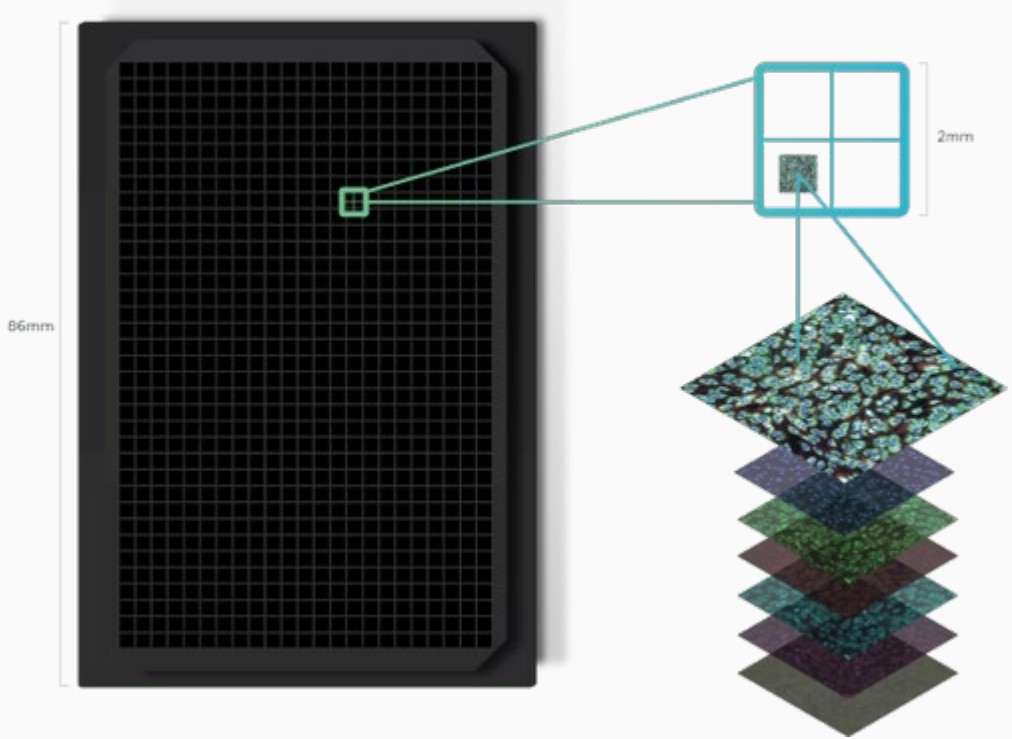
- 125,510 images of human cells, four cell types
  - U2OS: Bone cancer cells
  - RPE: Retinal Pigment Epithelium cells
  - HUVEC: Human Umbilical Vein cells
  - HEPG2: Human liver cancer cells
- Cells undergone knock-down treatment with siRNAs
- 1108 classes, one for each siRNA
- First (and 13’th) places reached over 0.99 accuracy
- https://www.kaggle.com/c/recursion-cellular-image-classification

Huge Batch Effect Makes The Problem Even Harder



From rxrx.ai, these images demonstrate the batch effects over two different classes

Plate, Experiment, Well Jargon



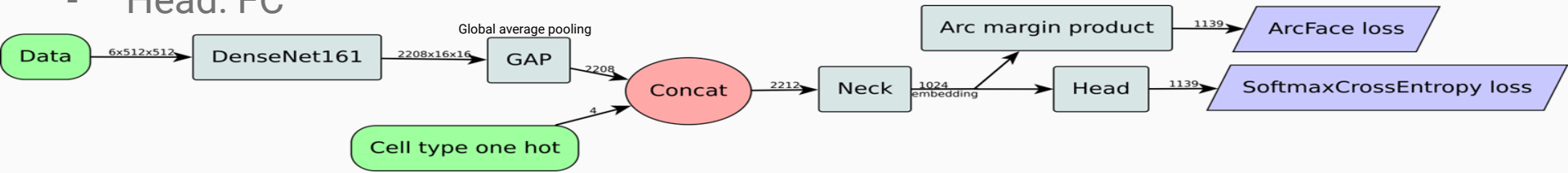
A 384 well plate and a 6-channel image

The 6 channels illuminate different organelles of the cell:

- The nucleus
- Endoplasmic reticulum
- Actin cytoskeleton
- Nucleolus
- Mitochondria
- Golgi apparatus

Overview of Kaggle First Place Winner - Bag of Tricks

- Data augmentation: flip, rotate, crop, random resize and normalization
- Cutmix: regional dropout (https://arxiv.org/abs/1905.04899)
- Using EfficientNet (https://arxiv.org/abs/1905.11946)
- Pseudo labeling and Linear Sum Assignment (LSA)
- Softmax + ArcFace (Additive Angular Margin) loss (https://arxiv.org/abs/1801.07698)
- Cosine learning rate decay
- Nvidia apex.amp (16 bit floating point magic)
- Neck: BN + FC + ReLU + BN + FC + BN
- Head: FC



https://www.kaggle.com/c/recursion-cellular-image-classification/discussion/110335

https://www.kaggle.com/c/recursion-cellular-image-classification/discussion/110543#latest-648051

https://arxiv.org/abs/1905.04899

https://arxiv.org/abs/1905.11946

https://arxiv.org/abs/1908.02983

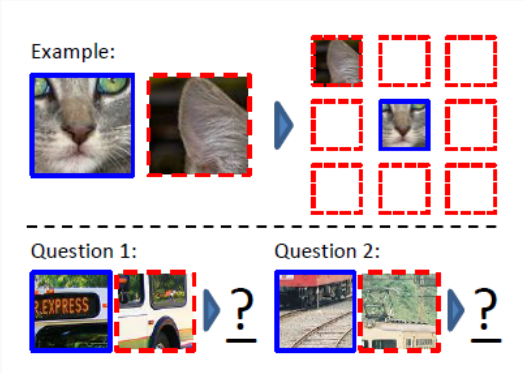
https://arxiv.org/abs/1801.07698

https://arxiv.org/abs/1806.01593v2

https://github.com/NVIDIA/apex

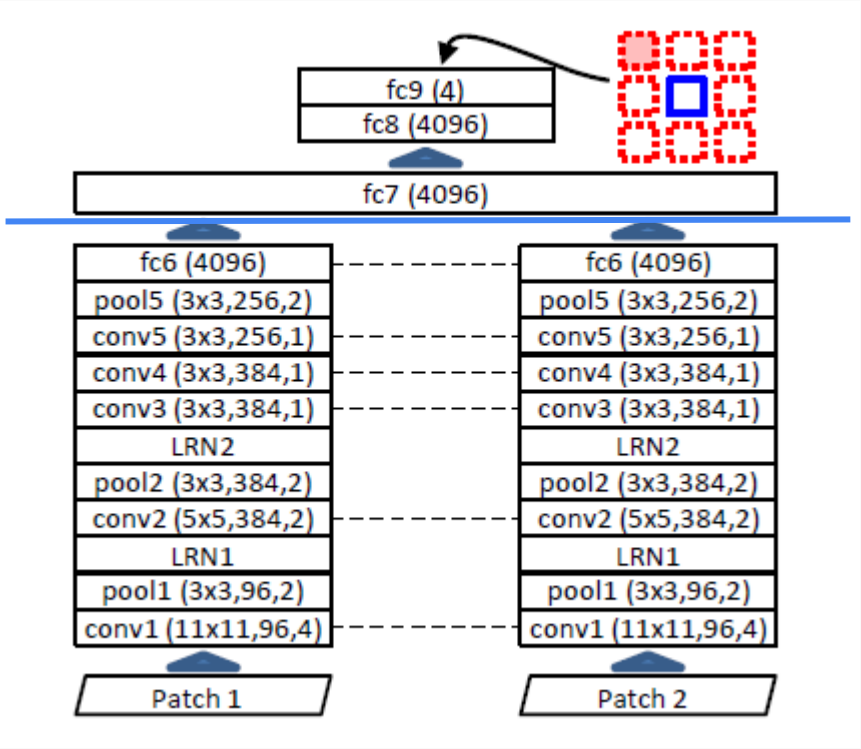
Project Plan - From Self Supervised to Supervised Classification of Cell Types

- Follow ideas of “Unsupervised Visual Representation Learning by Context Prediction” / Doerch, Gupta, Efros
  - They were able to achieve unsupervised object detection on ImageNet that is only 8% worse than the best R-CNN availble
- Train a self supervised model to recognize patch relative location
- Use the trained model core layers for **cell type classification**. Assumption is that semantic features on different cells are in core layers
- Hence we turn the problem from semi-supervised to supervised!

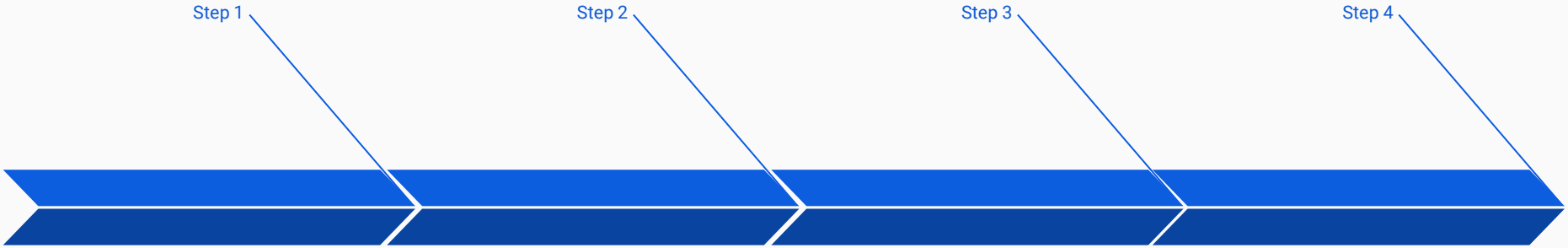


Project Plan: From Self Supervised to Supervised Cell Type Detection

- Idea is to use core layers for cell type classification
- This is kind of “transfer” learning
- Cut top layers (that focus on patching)
- Investigate core layers, to see if they are able to differentiate cell types



Project Overview



Data Preparation

Balance classes
Patching
Lazy label preparation

Model Build

'Dual network', select LR, optimizer, loss etc.

Unsupervised Learning

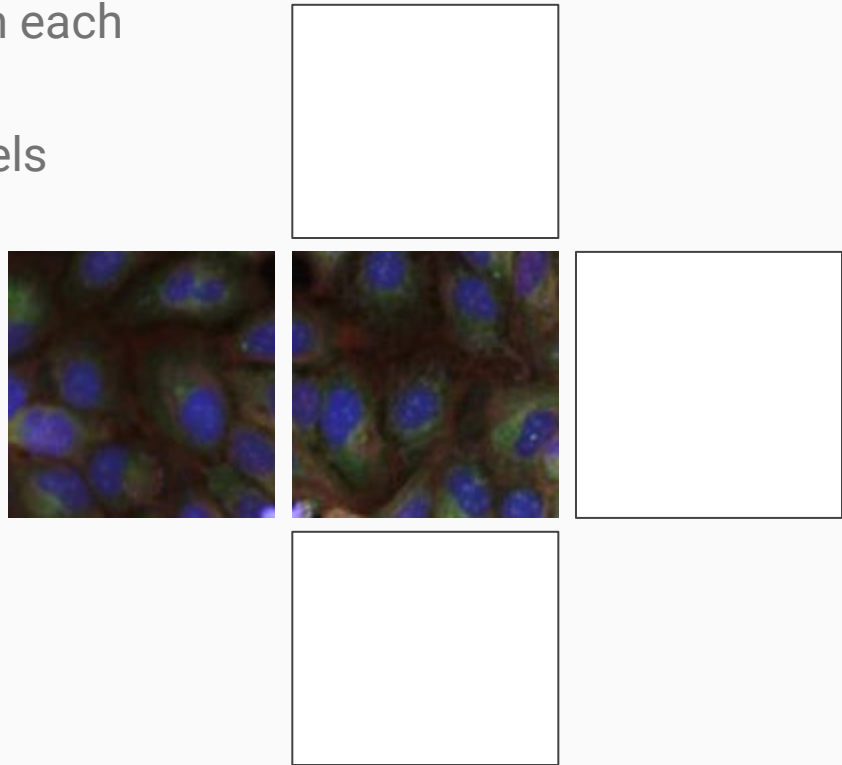
Optimize layers
Hyper parameter tuning
Patch location
Accuracy 0.9

Supervised Clustering Methods

tSNE
UMAP
Auto encoder

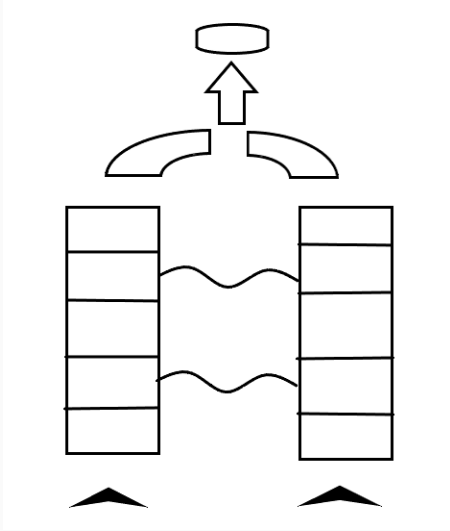
Data Preparation

- Resize images from 512x512 to 400x400 pixels
- Class balancing. In total we got 160000 images
- Select ‘center’ and a random side patch from each image
- Patches are randomly sliced to 133x133 pixels
- 4 classes: up, down, right, left
- Label is attached to the dataset
- Use lazy evaluation (Tensorflow TFRecord)
- Train and validation split



Model Build

- Batch size: 16
- After training with many options we used SGD optimizer, nesterov=True
- Loss: categorical cross entropy
- Labels are one-hot encoded
- We tried many different network architectures



ConvNet Siamese Network Architecture

Layer (type)	Output Shape	Param #		
Input_1	133, 133, 3	0		
Input_2	133, 133, 3	0		
Batch_normalization	133, 133, 3	12		
Conv2d + Relu padding='valid'	66, 66, 64	1792	filters=64, kernel_size=(3, 3), strides=(2, 2)	
Dropout	66, 66, 64	0		
Batch_normalization	66, 66, 64	256		
Conv2d + Relu padding='valid'	32, 32, 16	9232	filters=16, kernel_size=(3, 3), strides=(2, 2),	
Dropout	32, 32, 16	0		
Batch_normalization	32, 32, 16	64		
Conv2d + Relu padding='valid'	15, 15, 4	580	filters=4, kernel_size=(3, 3), strides=(2, 2),	
Batch_normalization	15, 15, 4	16		
Add	15, 15, 4	0		
Max_pooling2d	5, 5, 4	0		
Flatten	100	0		
Dense	16	1616		
Batch_normalization	16	64		
Dense	16	272		
Batch_normalization	16	64		
Dense	4	68		

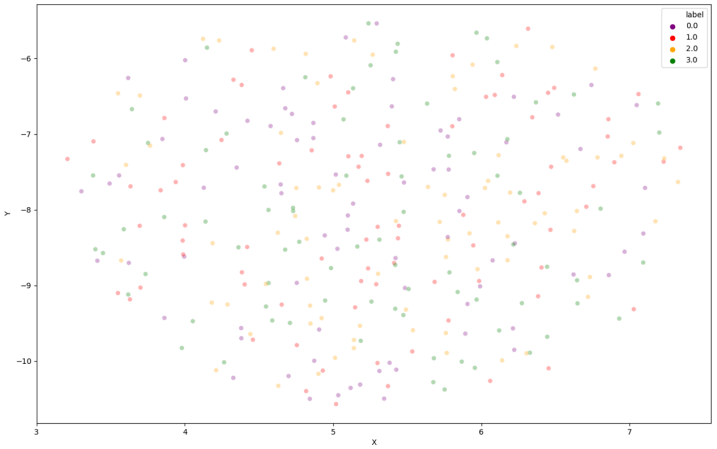
Total params: 14,036
Trainable params: 13,798
Non-trainable params: 238

Training for Patch Location

- A simple test of a similar network using supervised training for cell type quickly converged to above 0.9 accuracy
- Classes are: Up, Down, Right, Left
- Use 1 x GeForce GTX Titan GPU, 16 GB RAM
- Accuracy after 10 hours is above 0.9

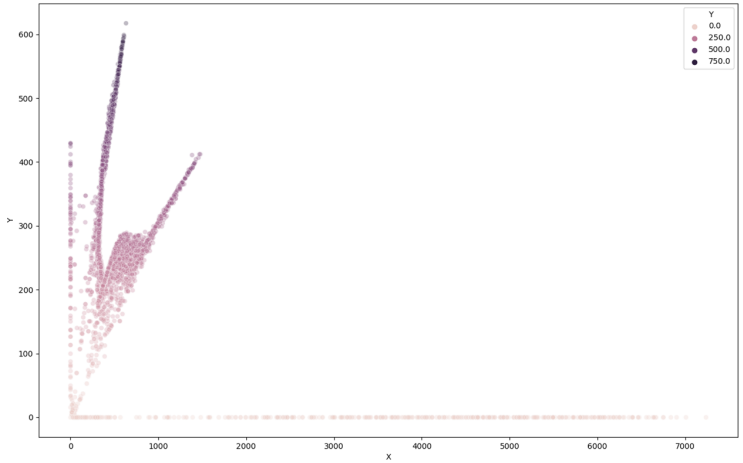
Use Trained Model Core Layers For Self Supervised Learning

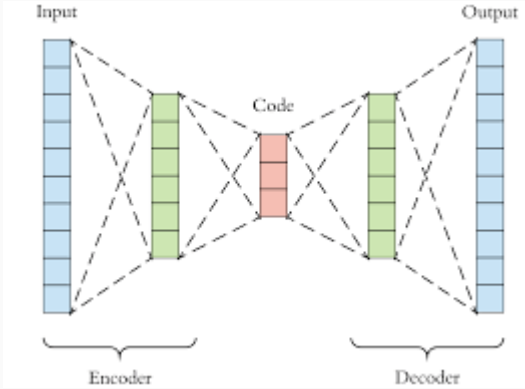
- Clustering on internal layers for cell type classification
- We tried many different clustering algorithms using outputs of internal layers:
  - PCA
  - tSNE
  - UMAP
  - DBSCAN
  - Spectral Clustering
  - Auto encoders
- **None really provided a convincing results**
- Something is not working :(.....



AutoEncoder for Dimension Reduction - Did Not Work

```
input_layer = Input(900)
encoder = BatchNormalization()(Dense(64,activation='relu')(input_layer))
encoder = BatchNormalization()(Dense(16, activation='relu')(encoder))
encoder = BatchNormalization()(Dense(2, activation='relu')(encoder))
decoder = BatchNormalization()(Dense(16, activation='relu')(encoder))
decoder = BatchNormalization()(Dense(64, activation='relu')(decoder))
decoder = BatchNormalization()(Dense(900, activation='relu')(decoder))
```



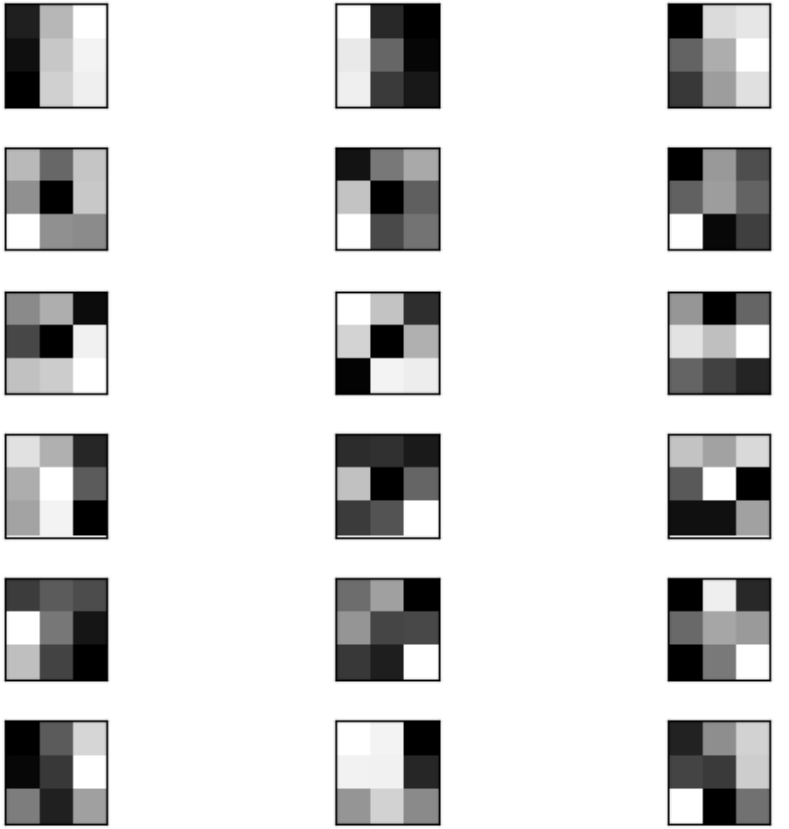


Something is Not Working

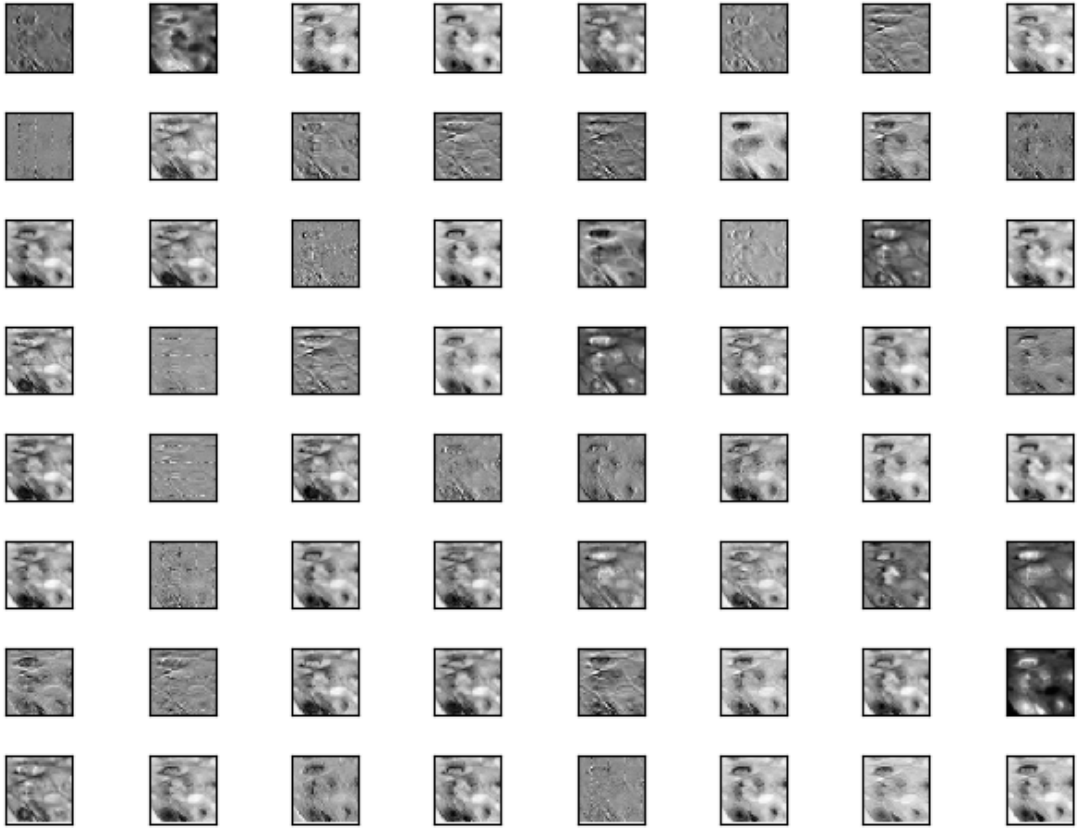
- Need to debug internal layers to try to see why clustering doesn’t work
- Check if are able to differentiate cell types
- Use feature vectors and feature maps

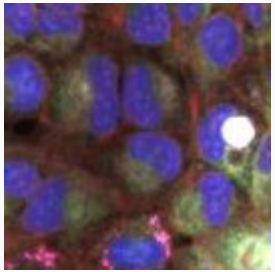


Visualize Feature Vectors



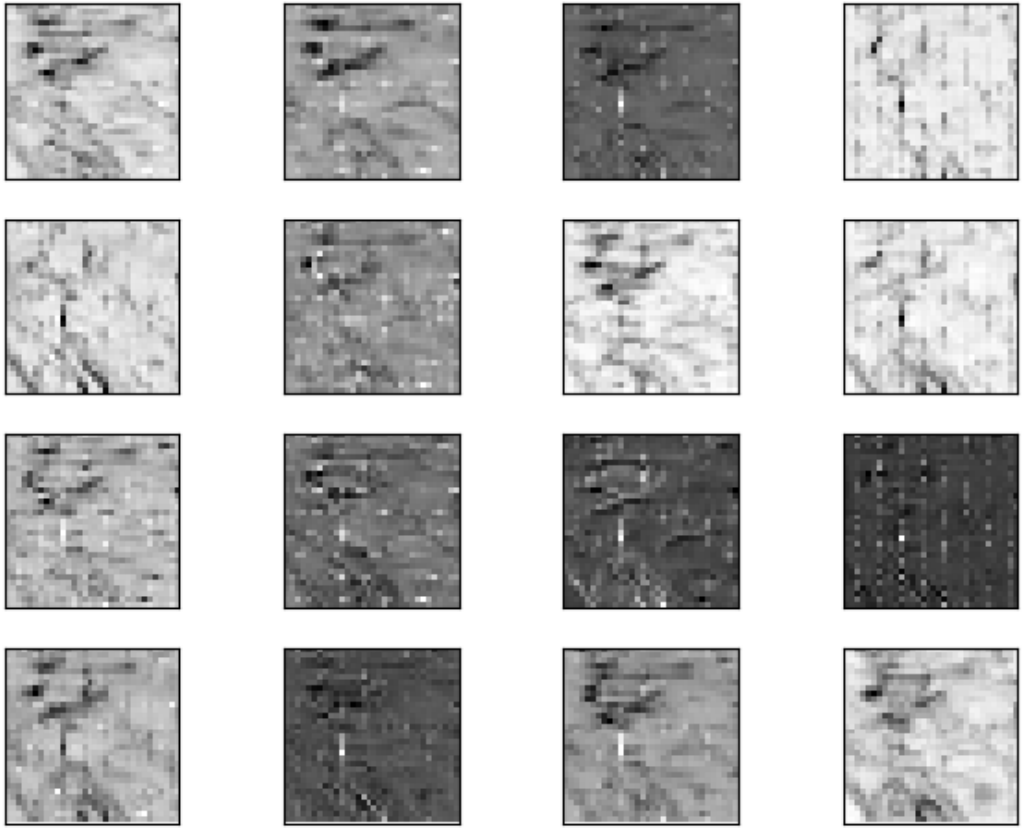
Visualize Feature Maps Conv 1

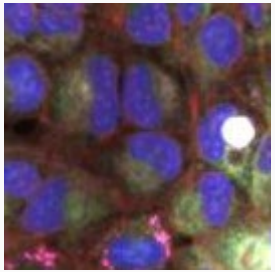




U2OS-05_4_G17_s1.jpg

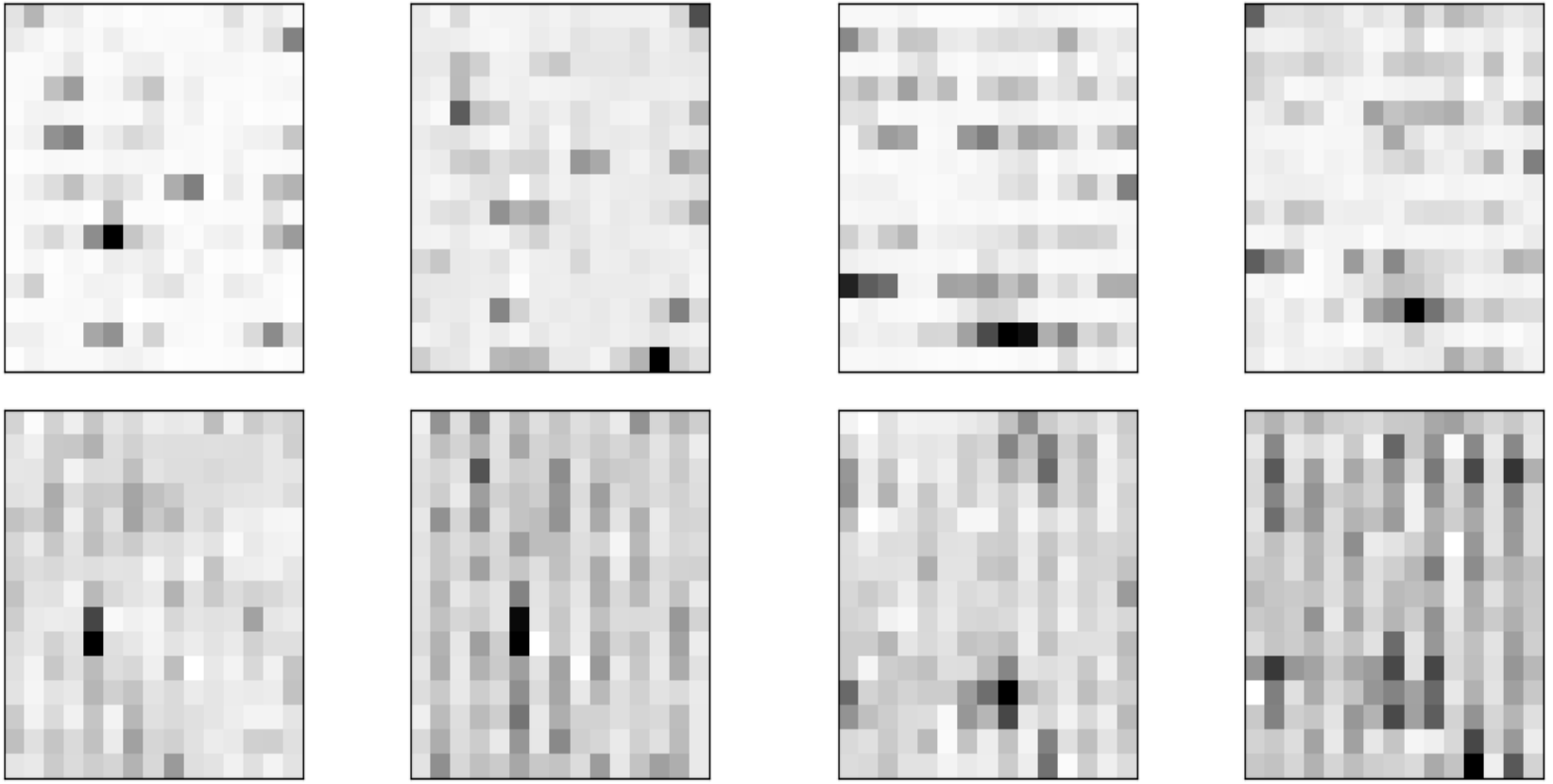
Visualize Feature Maps Conv2





U2OS-05_4_G17_s1.jpg

Feature Map Conv3 - RPE vs. U2OS Cell Types

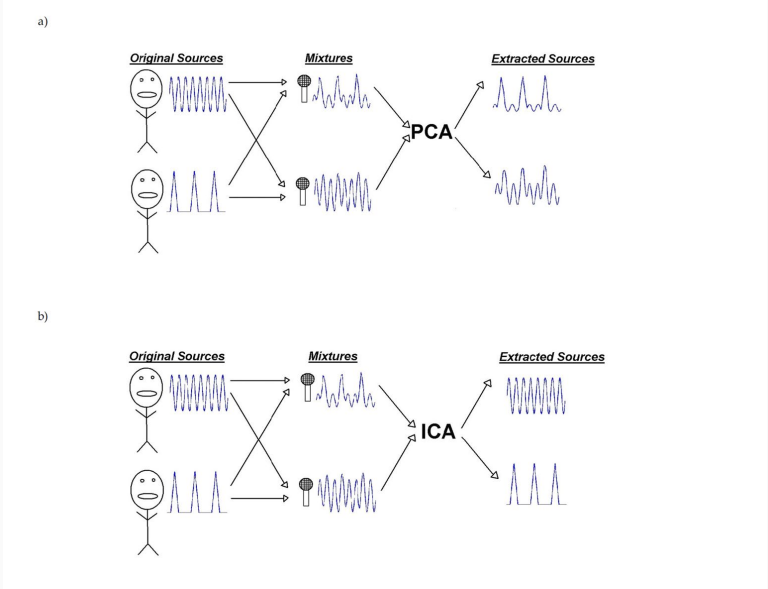


Improve Model

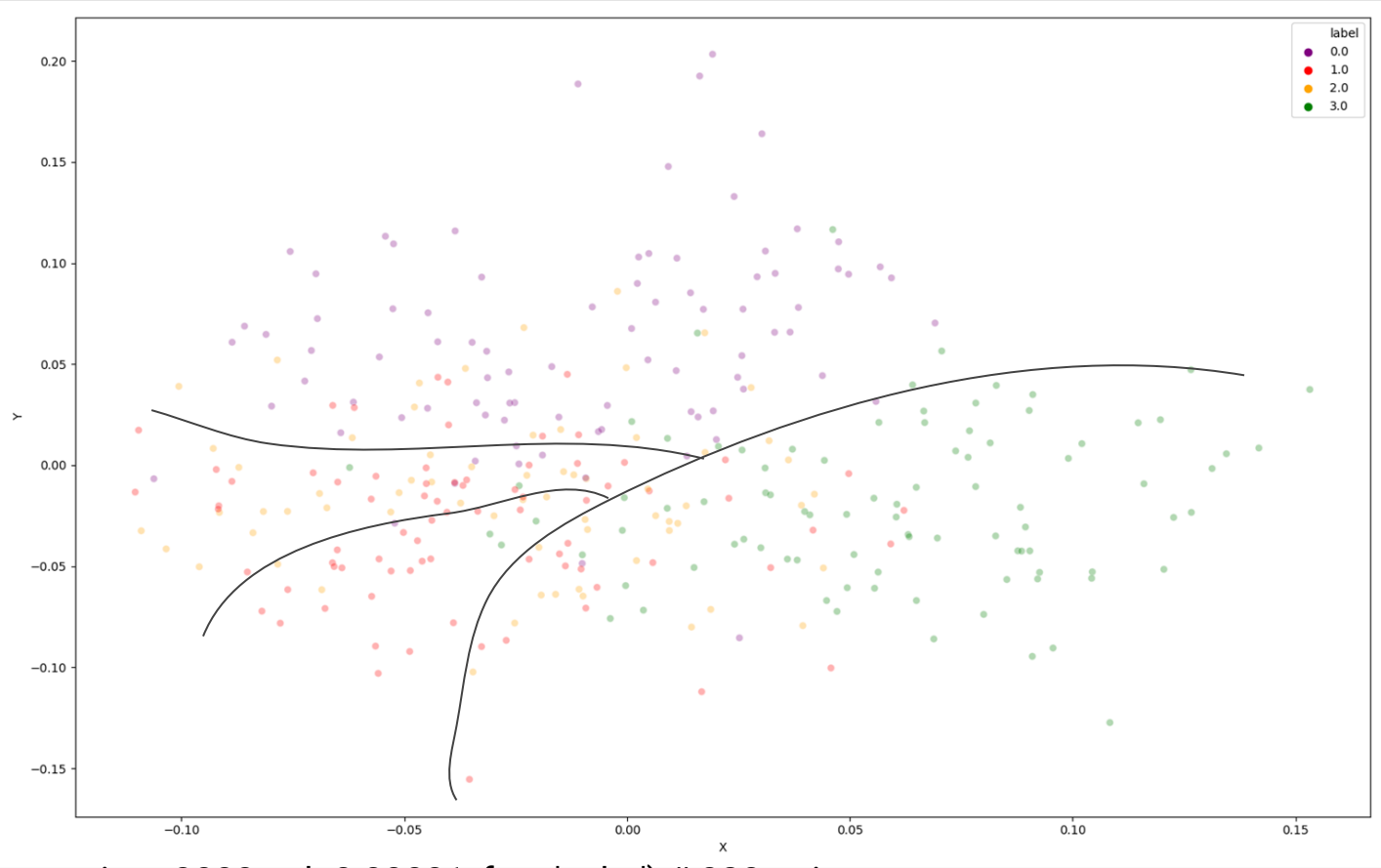
- We realized that out model is not specializing on cell types
- **Instead it identifies edges, color etc.**
  - We need to make the edge detection task harder
  - We cropped 15 pixels horizontally and vertically from each image
- Retraining again...
  - This time it took two days to converge to accuracy > 0.6
- Still, testing with our set of clustering methods still did not provide a convincing results :(

Finally - Fast ICA to The Rescue

- Independent Component Analysis
- A method for matrix factorization to independent components (similar to deconvolution)
- X(mixtures) = A(proportions) x S(sources)
- Requirements:
  - Sources are independent which can be achieved by
  - Minimization of mutual information or
  - Maximization of non-Gaussianity
- https://towardsdatascience.com/paper-summary-an-introduction-to-independent-component-analysis-infomax-and-fastica-algorithms-7b44d18ab393



FastICA Results



FastICA(2, max_iter=3000, tol=0.00001, fun='cube') # 320 points.

Summary

- Use a lesser deep model due to lesser number of objects to detect (4 types of cells as opposed to natural images)
- Using a large variety of clustering approaches
- Tested different architectures and parameters



Thanks!

